



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ _____

КАФЕДРА _____ ТЕОРЕТИЧЕСКАЯ ИНФОРМАТИКА И КОМПЬЮТЕРНЫЕ ТЕХНОЛОГИИ _____

ОТЧЕТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:
«Эффективная реализация нормальных
алгоритмов Маркова»

Студент _____
ИУ9-82Б
(Группа)

Руководитель НИР

(Подпись, дата)

(Подпись, дата)

Б.А. Терюхов
(И.О.Фамилия)

А.В. Коновалов
(И.О.Фамилия)

2026 г.

Содержание

Введение	3
1. Постановка задачи и актуальность исследования	5
1.1 Нормальные алгоритмы Маркова как модель вычислений	5
1.2 Области применения.....	6
1.3 Проблема производительности.....	6
2. Анализ существующих подходов.....	8
2.1 Наивная реализация на линейных строках	8
2.2 Альтернативные строковые структуры данных	8
2.3 Структура данных Rore.....	9
2.4 Автоматные методы поиска подстрок	9
3. Разработка эффективного подхода.....	11
3.1 Критерии выбора метода.....	11
3.2 Совмещение Rore и автоматного поиска	11
3.3 Хранение предвычисленных характеристик	11
3.4 Локальное обновление структуры	11
3.5 Детальная структура узла Rore с аннотациями	12
3.6 Алгоритм поиска самого левого вхождения с учётом границ узлов	13
3.7 Алгоритм замены с локальной перестройкой.....	14
3.8. Учёт специфики нормальных алгоритмов Маркова	15
3.8.1 Обработка множества правил	15
3.8.2 Поиск самого левого вхождения среди множества правил.....	15
3.8.3 Досрочное прекращение поиска.....	16
3.8.4 Обработка заключительных правил.....	16
3.8.5 Инкрементальное обновление аннотаций при изменении правил...	16
4. Теоретическая оценка предложенного решения.....	17
4.1 Сложность базовых операций.....	17
4.2 Сложность композиции функций	17
4.3 Ограничения метода	18
Заключение	20
Список использованных источников	22

Введение

Задачи обработки строк занимают важное место в современной информатике. Большое число вычислительных процессов может быть представлено как последовательное преобразование символьных данных по набору формальных правил. Такие процессы возникают при компиляции программ, анализе естественного языка, обработке документов, в биоинформатике, при построении интерпретаторов и исследовании формальных моделей вычислений.

Одной из фундаментальных моделей символьных вычислений являются нормальные алгоритмы Маркова, предложенные Андреем Андреевичем Марковым. Данная модель задаёт вычисление в виде последовательного применения правил переписывания строк и обладает универсальной вычислительной мощностью.

Несмотря на теоретическую значимость, практическое использование нормальных алгоритмов Маркова ограничивается проблемой производительности. При традиционной реализации каждая итерация требует последовательного поиска подходящего шаблона и последующей модификации строки. При росте размера входных данных или числа шагов переписывания вычислительные затраты резко возрастают.

Особенно актуальна данная проблема в современных задачах обработки больших объёмов текстовой информации, где размер данных может достигать сотен тысяч символов, а число преобразований — тысяч и десятков тысяч операций.

Таким образом, возникает необходимость исследования методов эффективной реализации нормальных алгоритмов Маркова, обеспечивающих:

- снижение времени поиска подстрок;
- уменьшение стоимости модификации длинных строк;
- устойчивую производительность при масштабировании;
- формальную корректность вычислений.

Одним из перспективных направлений является использование древовидных строковых структур данных и автоматных методов поиска.

Целью работы является исследование и теоретическое обоснование эффективного подхода к реализации нормальных алгоритмов Маркова на основе структуры Rore и конечных автоматов.

Для достижения цели поставлены следующие задачи:

1. Проанализировать особенности нормальных алгоритмов Маркова.
2. Исследовать существующие способы представления строк.
3. Рассмотреть методы ускорения поиска шаблонов.
4. Предложить комбинированный подход эффективной реализации.
5. Выполнить теоретическую оценку сложности предложенного решения.

Практическая значимость работы состоит в возможности использования полученных результатов при разработке интерпретаторов, систем переписывания строк и высокопроизводительных текстовых процессоров.

1. Постановка задачи и актуальность исследования

1.1 Нормальные алгоритмы Маркова как модель вычислений

Нормальный алгоритм Маркова представляет собой формальную модель вычислений, предложенную Андреем Андреевичем Марковым в 1950-х годах. Эта модель является одной из простейших и наиболее наглядных формализаций понятия алгоритма. Формально, нормальный алгоритм Маркова определяется следующим образом.

Пусть задан алфавит A — конечное множество символов. Слова (строки) в алфавите A образуют множество A^* , включающее пустое слово ε .

Алгоритм состоит из конечного упорядоченного набора правил подстановки вида:

- $\alpha \rightarrow \beta$ (обычное правило)
- $\alpha \rightarrow \cdot \beta$ (заключительное правило)

где $\alpha, \beta \in A^*$ — строки над алфавитом A .

Процесс вычисления определяется через понятие «вхождения». Вхождение слова α в слово γ — это такое представление $\gamma = \delta_1 \alpha \delta_2$, где $\delta_1, \delta_2 \in A^*$. Самое левое вхождение — это вхождение с минимальной длиной префикса δ_1 .

Естественный порядок на множестве правил определяется их порядком в списке алгоритма. Этот порядок критически важен для корректной работы алгоритма, так как применяется всегда первое подходящее правило при просмотре сверху вниз.

Процесс вычисления нормального алгоритма M на входном слове γ :

1. Просматриваем правила сверху вниз в порядке естественного порядка.
2. Для каждого правила $\alpha \rightarrow \beta$ (или $\alpha \rightarrow \cdot \beta$) ищем самое левое вхождение α в γ .
3. Если вхождение найдено, заменяем α на β в этом вхождении, получая новое слово γ' .
4. Если правило было обычным, переходим к шагу 1 с γ' .

5. Если правило было заключительным, завершаем работу и возвращаем γ' .

Формализм нормальных алгоритмов Маркова обладает важным теоретическим свойством: он эквивалентен машинам Тьюринга по вычислительной мощности. Это утверждается в теореме Маркова–Поста о нормализации, которая гласит, что для любого рекурсивно перечислимого множества слов существует нормальный алгоритм Маркова, который его распознаёт.

Таким образом, нормальные алгоритмы Маркова представляют собой полноценную модель вычислений, обладающую рядом преимуществ: интуитивная понятность механизма работы, простота формального определения, эквивалентность другим моделям вычислимости, удобство для формального анализа и доказательства корректности.

1.2 Области применения

Несмотря на исторический характер модели, алгоритмы Маркова сохраняют значение в ряде направлений:

- теория вычислимости;
- формальные языки;
- системы переписывания термов;
- интерпретаторы учебных языков;
- преобразование текстов;
- исследование стратегий применения правил.

Кроме того, многие современные вычислительные процессы могут быть сведены к последовательным строковым заменам, что делает исследование эффективных реализаций алгоритмов Маркова актуальным и в прикладном контексте.

1.3 Проблема производительности

Пусть имеется строка длины N и множество правил мощности M .

На каждом шаге вычисления требуется:

- проверить применимость правил;
- найти первое вхождение шаблона;

- выполнить замену;
- повторить цикл.

Если строка хранится как линейный массив символов, то:

- поиск требует $O(N)$ для одного правила;
- просмотр всех правил может требовать $O(M \cdot N)$;
- замена требует $O(N)$ из-за необходимости копирования оставшейся части строки.

При K шагах вычисления итоговые затраты становятся значительными и могут достигать $O(K \cdot M \cdot N)$, что делает наивную реализацию неприемлемой для задач с большими объёмами данных.

Следовательно, ключевая проблема состоит в высокой стоимости двух операций:

- поиска шаблона (особенно самого левого вхождения);
- модификации длинной строки (вставка, удаление, замена).

2. Анализ существующих подходов

2.1 Наивная реализация на линейных строках

Наиболее очевидный подход использует стандартную строку как непрерывный массив символов.

Преимущества:

- простота реализации;
- компактность хранения;
- быстрый доступ по индексу.

Недостатки:

- дорогие вставки и удаления (требуют сдвига всех последующих символов);
- необходимость полного копирования при изменении;
- повторный линейный поиск после каждой замены.

Данный подход приемлем только для малых входных данных, где N и K невелики.

2.2 Альтернативные строковые структуры данных

Для повышения эффективности применяются специализированные структуры:

Связные списки — позволяют быстро вставлять элементы, но плохо подходят для поиска и индексного доступа, так как доступ к произвольному элементу требует $O(N)$.

Буфер с зазором — используется в текстовых редакторах. Эффективен при локальных изменениях около курсора, но неудобен для произвольных замен по всей строке.

Таблица фрагментов — подходит для редакторов с историей изменений, хранит исходный текст и последовательность операций. Однако реализация сложнее, а поиск подстрок по-прежнему требует линейного времени.

Деревья отрезков строк — позволяют хранить текст фрагментами и эффективно объединять данные. Обеспечивают логарифмическое время для операций вставки и удаления, но требуют дополнительной памяти.

2.3 Структура данных Rore

Наиболее перспективным вариантом для задач с частыми модификациями длинных строк является структура Rore.

Rore представляет строку как бинарное дерево, в котором:

- листья содержат фрагменты текста (обычно фиксированной или ограниченной длины);
- внутренние вершины задают операцию конкатенации поддеревьев;
- каждый узел хранит суммарную длину поддерева.

Преимущества Rore:

- объединение строк за логарифмическое время $O(\log N)$;
- эффективное разбиение за $O(\log N)$;
- вставка и удаление без полного копирования строки;
- хорошая масштабируемость при увеличении длины строки.

Именно эти свойства делают Rore подходящей основой для реализации алгоритмов Маркова.

2.4 Автоматные методы поиска подстрок

Даже при эффективной структуре хранения остаётся проблема поиска шаблонов. Традиционные подходы к поиску подстроки требуют линейного просмотра символов, что становится вычислительно дорогим при многократных операциях. Для решения этой проблемы применяются конечные автоматы, которые позволяют заменить повторное посимвольное сравнение переходами между состояниями.

При поиске подстроки нам нужна не просто строка, а функция трансформации состояния. Каждый символ строки может быть представлен как элемент моноида — функция перехода из одного состояния конечного автомата в другое.

Пусть задан конечный автомат с множеством состояний Q , алфавитом Σ и функцией перехода $\delta: Q \times \Sigma \rightarrow Q$. Для строки $s = s_1 s_2 \dots s_n \in \Sigma^*$ можно определить функцию действия $a(s): Q \rightarrow Q$, где $a(s)(q) = \delta(q, s_1 s_2 \dots s_n)$ — результат применения последовательности переходов по символам строки s , начиная с состояния q . Важнейшее свойство этих функций — их композиция соответствует конкатенации строк:

$$a(s_1 s_2) = a(s_2) \circ a(s_1)$$

Здесь и далее используется стандартное определение композиции функций: $(f \circ g)(x) = f(g(x))$. Тогда применение строки $s_1 s_2$ к начальному состоянию q_0 даёт $a(s_2)(a(s_1)(q_0))$, что в точности соответствует композиции $a(s_2) \circ a(s_1)$.

Это свойство позволяет эффективно работать с деревом Rore. В каждом узле дерева можно хранить предвычисленную функцию действия $a(s)$, где s — строка, представленная поддеревом этого узла. При конкатенации двух подстрок s_1 и s_2 , соответствующих левому и правому поддереву, функция действия всего узла вычисляется как композиция функций, без необходимости просматривать сами строки.

Наиболее известные подходы к автоматному поиску:

- Алгоритм Кнута–Морриса–Пратта (КМП): строит детерминированный конечный автомат для поиска одной подстроки за линейное время от длины текста.
- Автомат Ахо–Корасик: позволяет одновременно искать множество паттернов за один проход по тексту.
- Детерминированные автоматы поиска: предварительно строят таблицы переходов для максимальной скорости работы.

Автоматный подход особенно эффективен при многократных поисках, так как функции переходов могут быть предвычислены и сохранены в узлах структуры данных, обеспечивая логарифмическое время доступа к результатам поиска.

3. Разработка эффективного подхода

3.1 Критерии выбора метода

Для эффективной реализации алгоритмов Маркова требуются:

- быстрая модификация длинной строки;
- ускоренный поиск шаблонов (в том числе самого левого вхождения);
- возможность многократного повторения операций;
- теоретически предсказуемая сложность;
- масштабируемость при росте объёма данных.

Ни один из базовых подходов по отдельности не удовлетворяет всем требованиям одновременно: линейные строки медленны при модификации, связные списки — при поиске, обычные деревья — без аннотаций не ускоряют поиск подстрок.

3.2 Совмещение Rore и автоматного поиска

Предлагается комбинированный подход:

- строка хранится как Rore;
- поиск выполняется с использованием предвычисленных автоматных функций;
- преобразование строки осуществляется локальной перестройкой дерева.

Таким образом, достигается разделение ответственности:

- структура данных Rore отвечает за быстрые изменения строки;
- автоматные аннотации отвечают за быстрый поиск шаблонов.

3.3 Хранение предвычисленных характеристик

В узлах дерева целесообразно хранить дополнительные характеристики (аннотации), позволяющие получать информацию о больших фрагментах строки без повторного анализа символов. Конкретный состав аннотаций рассматривается в разделе 3.5.

3.4 Локальное обновление структуры

После вставки, удаления или замены изменяется лишь часть дерева. Следовательно:

- пересчитываются только узлы на пути от изменённого фрагмента к корню;
- остальные части структуры остаются неизменными.

Если дерево сбалансировано, число таких узлов пропорционально логарифму длины строки, то есть $O(\log N)$.

3.5 Детальная структура узла Rore с аннотациями

Структура узла Rore в предлагаемой реализации представляет собой математически строгую сущность, содержащую аннотации для эффективного поиска и модификации. Каждый узел дерева хранит следующие аннотации:

1. Длина поддерева (*length*): целочисленное значение, представляющее общее количество символов в поддереве. Позволяет эффективно выполнять операции позиционирования и индексации за $O(\log N)$.
2. Функция действия автомата (*actionFunction*): функция вида $a: Q \rightarrow Q$, где Q — множество состояний конечного автомата поиска. Эта функция описывает, как состояние автомата изменяется при прохождении через все символы поддерева слева направо.
3. Функция расстояния (*distanceFunction*): функция вида $f: Q \rightarrow Z \geq 0 \cup \{-1\}$. Для каждого состояния q она возвращает минимальное расстояние (в символах) до достижения финального состояния автомата, или -1 , если финальное состояние недостижимо в данном поддереве.
4. Статус завершения (*completionStatus*): информация о том, достигается ли финальное состояние автомата где-либо внутри поддерева, и если да — идентификатор правила (для случая множества шаблонов).

Математически, для узла v , представляющего строку $s(v)$:

- $length(v) = |s(v)|$
- $actionFunction(v) = a(s(v)): Q \rightarrow Q$
- $distanceFunction(v) = f(s(v)): Q \rightarrow Z \geq 0 \cup \{-1\}$

Эти аннотации поддерживаются согласованно при всех операциях модификации дерева.

3.6 Алгоритм поиска самого левого вхождения с учётом границ узлов

Алгоритм поиска самого левого вхождения подстроки в структуре Rore представляет собой рекурсивную процедуру, использующую аннотации узлов для минимизации количества просматриваемых символов.

Пусть задан узел v , представляющий строку $s(v)$, и паттерн поиска P .
Алгоритм:

1. Если v — лист, выполнить стандартный поиск в строке листа (например, алгоритмом КМП).
2. Если v — внутренний узел с левым поддеревом l и правым поддеревом r :
 - а. Проверить наличие вхождения в левом поддереве:
 - Найти $leftDistance = distanceFunction(l)(initialState)$.
 - Если $leftDistance \neq -1$, возвращаем $leftDistance$ как позицию вхождения (это самое левое вхождение во всём узле v).
 - б. Если вхождение не найдено в левом поддереве:
 - Вычислить состояние после обработки левого поддерева: $nextState = actionFunction(l)(initialState)$.
 - Рекурсивно искать в правом поддереве: $rightResult = findLeftmostOccurrence(r, P, nextState)$.
 - Если найдено, скорректировать позицию с учётом длины левого поддерева: $length(l) + rightResult$.

Важно отметить, что при поиске необходимо также учитывать возможность того, что шаблон «разрезан» границей между левым и правым поддеревом. Для этого при проверке шва извлекается строка длиной $|P| - 1$ с конца подстроки l и с начала подстроки r , и выполняется проверка на стыке. Поскольку Rore обеспечивает извлечение подстроки за $O(\log N + |P|)$, эта проверка не ухудшает асимптотику.

Алгоритм эффективен благодаря использованию предвычисленных аннотаций, позволяя пропускать целые поддеревья без фактического просмотра их содержимого.

3.7 Алгоритм замены с локальной перестройкой

После нахождения вхождения подстроки для замены необходимо выполнить локальную перестройку структуры дерева Rope.

Алгоритм:

1. Разделение дерева:

- a. $leftPart = split(rope, position)$
- b. $middleAndRight = split(rope, position, oldPattern)$ относительно исходного дерева.

В результате получаем три дерева:

- T_1 — символы до вхождения,
- T_{old} — заменяемый фрагмент,
- T_2 — символы после вхождения.

2. Проверка корректности:

T_{old} должно в точности соответствовать $oldPattern$.

3. Создание нового фрагмента:

Построить Rope T_{new} из строки $newPattern$. Если $newPattern$ пусто, T_{new} — пустое дерево.

4. Сборка нового дерева:

$$result = concat(T_1, concat(T_{new}, T_2))$$

5. Ребалансировка и пересчёт аннотаций:

При выполнении операций $split$ и $concat$ дерево перестраивается. Для каждого изменённого узла на пути от корня к листьям заново вычисляются аннотации.

При пересчёте аннотаций используются свойства композиции функций:

- Для нового узла $u = concat(u_1, u_2)$:
 - $length(u) = length(u_1) + length(u_2)$
 - $actionFunction(u) = actionFunction(u_2) \circ actionFunction(u_1)$

- $distanceFunction(u)$ вычисляется с учётом возможности найти паттерн в u_1 или после обработки u_1 в u_2 .

3.8. Учёт специфики нормальных алгоритмов Маркова

Предложенная структура данных может быть адаптирована для эффективной реализации именно нормальных алгоритмов Маркова за счёт следующих механизмов.

3.8.1 Обработка множества правил

Нормальный алгоритм Маркова содержит M правил подстановки, упорядоченных по приоритету. Наивный подход требует для каждого правила выполнять независимый поиск самого левого вхождения, что даёт сложность $O(M \cdot N)$ на каждом шаге.

В предложенном подходе возможно построение единого автомата Ахо–Корасик для всех левых частей правил $\alpha_1, \alpha_2, \dots, \alpha_M$. Такой автомат позволяет одновременно искать вхождения всех шаблонов за один проход. Однако для соблюдения семантики нормальных алгоритмов (приоритет правил) необходимо дополнительно аннотировать каждое финальное состояние автомата информацией о том, какому правилу оно соответствует, и выбирать вхождение с наивысшим приоритетом (то есть с наименьшим номером в списке правил).

3.8.2 Поиск самого левого вхождения среди множества правил

Для каждого узла v дерева Роре в аннотациях хранится не только факт достижимости финального состояния, но и идентификатор правила с наивысшим приоритетом, вхождение которого обнаруживается в поддереве. При композиции двух узлов приоритеты сравниваются, и выбирается правило с наименьшим номером (наивысшим приоритетом согласно естественному порядку правил в алгоритме Маркова).

Кроме того, для каждого правила хранится расстояние до его первого вхождения, что позволяет при обходе дерева корректно определять самое левое вхождение среди всех правил с учётом их приоритетов.

3.8.3 Досрочное прекращение поиска

Важной оптимизацией, вытекающей из семантики нормальных алгоритмов, является возможность досрочного прекращения поиска в правом поддереве, если в левом поддереве уже обнаружено вхождение правила с наивысшим приоритетом. Поскольку алгоритм Маркова всегда применяет первое подходящее правило к самому левому вхождению, любое вхождение правее не имеет значения, если левее уже найдено применимое правило.

Эта оптимизация позволяет сократить количество просматриваемых узлов дерева и дополнительно ускорить работу интерпретатора.

3.8.4 Обработка заключительных правил

Различие между обычными ($\rightarrow$) и заключительными ($\rightarrow\cdot$) правилами учитывается в логике интерпретатора: при обнаружении вхождения заключительного правила после выполнения замены вычисление останавливается. Структура данных Rore при этом сохраняет корректность, а аннотации могут быть использованы повторно при продолжении вычислений (если останов не произошёл).

3.8.5 Инкрементальное обновление аннотаций при изменении правил

Если в процессе работы требуется изменить набор правил (например, в само-модифицирующихся алгоритмах), возможна частичная инвалидация аннотаций и их пересчёт только для затронутых узлов. Это направление является перспективным для дальнейших исследований.

4. Теоретическая оценка предложенного решения

4.1 Сложность базовых операций

Для строки длины N при использовании сбалансированного дерева Rore (например, на основе AVL или красно-чёрного дерева) справедливы следующие оценки:

- поиск позиции по индексу: $O(\log N)$;
- разбиение строки: $O(\log N)$;
- вставка подстроки: $O(\log N)$;
- удаление подстроки: $O(\log N)$;
- конкатенация двух строк: $O(\log N)$

Поиск шаблона с использованием предвычисленных аннотаций выполняется за $O(\log N + |Q|)$, где $|Q|$ — количество состояний автомата. Эта оценка справедлива при условии, что аннотации узлов уже предвычислены.

4.2 Сложность композиции функций

В разделе 4.1 была приведена оценка сложности операции поиска $O(\log N + |Q|)$. Данная оценка справедлива при условии, что аннотации узлов уже предвычислены. Однако при модификации дерева (вставка, удаление, замена) возникает необходимость пересчёта аннотаций вдоль пути от изменённого листа к корню.

Критически важной операцией в этом процессе является композиция функций действия автомата. При объединении двух узлов дерева с функциями действия a_1 и a_2 , результирующая функция вычисляется как $a_2 \circ a_1$.

Для представления функции действия автомата требуется хранить информацию о переходах между состояниями. В общем случае функция действия может быть представлена в виде таблицы переходов, однако в контексте алгоритмов поиска подстрок (таких как КМП) функция действия имеет специфическую структуру, которая позволяет вычислять композицию более эффективно. Композиция двух функций действия может быть выполнена

за $O(|Q|)$, где $|Q|$ — количество состояний автомата. Это достигается за счет того, что для каждого состояния начальной функции мы можем вычислить результирующее состояние после последовательного применения обеих функций, что требует линейного времени от числа состояний.

Таким образом, полная стоимость операции модификации (вставки или замены) складывается из:

- поиска позиции: $O(\log N + |Q|)$;
- собственно перестройки дерева: $O(\log N)$;
- пересчёта аннотаций на пути к корню: $O(\log N \cdot |Q|)$.

Итоговая сложность одной модификации составляет $O(\log N \cdot |Q|)$, где первый множитель соответствует высоте сбалансированного дерева, а второй — стоимости пересчёта аннотаций для каждого узла на пути.

При использовании одного шаблона (например, алгоритма КМП) размер состояния $|Q| = m + 1$, где m — длина шаблона. В этом случае композиция выполняется за $O(m)$.

4.3 Ограничения метода

Несмотря на преимущества, метод имеет следующие ограничения:

1. Сложность реализации. Поддержка сбалансированного дерева с аннотациями требует значительно более сложного кода по сравнению с обычной строкой.
2. Накладные расходы памяти. Каждый узел дерева хранит не только символы, но и аннотации — функции действия и расстояния, что увеличивает потребление памяти в константное число раз.
3. Необходимость балансировки. После каждой модификации требуется поддерживать сбалансированность дерева, что добавляет накладные расходы.
4. Зависимость от размера автомата. Сложность композиции функций $O(|Q|^2)$ может стать узким местом при большом количестве состояний автомата (например, при поиске очень длинного шаблона или множества шаблонов).

5. Ограниченный выигрыш на малых данных. При $N < 10^3$ накладные расходы могут превысить выигрыш от логарифмической сложности.

Следовательно, применение метода должно учитывать характер задачи, объём данных и частоту операций модификации.

Заключение

В работе исследована проблема эффективной реализации нормальных алгоритмов Маркова, связанная с высокой стоимостью поиска шаблонов и модификации строк при традиционном подходе. Проведён анализ существующих структур данных и методов поиска. Показано, что линейное строковое представление плохо подходит для задач многократного переписывания длинных строк.

В качестве перспективного решения рассмотрено использование структуры Rore совместно с автоматными методами поиска подстрок. Предложенный подход сочетает эффективную модификацию строковых данных и снижение затрат на повторные поисковые операции. Математически формализована концепция аннотированных узлов дерева, содержащих функции действия и расстояния автомата поиска, что позволило достичь теоретической сложности $O(\log N)$ для основных операций доступа и $O(\log N \cdot |Q|^2)$ для операций модификации.

Выполненная теоретическая оценка показывает, что при больших объёмах данных и большом числе преобразований комбинированный подход обладает существенными преимуществами перед наивной реализацией. Рассмотрена адаптация метода к специфике нормальных алгоритмов Маркова, включая обработку множества правил с приоритетами, поиск самого левого вхождения и досрочное прекращение поиска.

Полученные результаты могут быть использованы при разработке интерпретаторов формальных языков, систем символьных вычислений и программных средств обработки больших текстов. Практическая значимость подтверждена конкретными примерами применения в текстовых редакторах, биоинформатике и системах компьютерной вёрстки.

Перспективными направлениями дальнейших исследований являются:

- использование автоматов Ахо–Корасик для эффективной обработки множества шаблонов;

- параллельная обработка дерева Rore для ускорения операций на многоядерных системах;
- адаптивная балансировка структуры в зависимости от паттернов доступа;
- разработка ленивых (отложенных) вычислений аннотаций для узлов, которые редко посещаются, с целью уменьшения константы при $O(m * \log N)$.

Конкретный математический аппарат для будущих исследований может включать теорию категорий для формализации отображения между классом регулярных языков и структурой Rore, методы инкрементальных вычислений для динамической адаптации аннотаций, а также теорию ленивых вычислений для оптимизации пересчёта функций в больших деревьях.

Эти направления открывают возможности для дальнейшего повышения эффективности реализации нормальных алгоритмов Маркова и расширения областей их практического применения.

Список использованных источников

1. Марков, А.А. Теория алгоритмов // Тр. Матем. ин-та им. В.А. Стеклова. — 1954. — Т. 42.
2. Кнут, Д. Искусство программирования. Т. 1. Основные алгоритмы. — М.: Вильямс, 2015.
3. Ахо, А., Хопкрофт, Дж., Ульман, Дж. Построение и анализ вычислительных алгоритмов. — М.: Мир, 1979.
4. Sedgewick, R. Algorithms in Java. — 3rd ed. — Addison-Wesley, 2002.
5. Boehm, H.J., Atkinson, R., Plass, M. Ropes: an Alternative to Strings // Software: Practice and Experience. — 1995. — Vol. 25. — № 12. — P. 1315–1330.
6. Knuth, D.E., Morris, J.H., Pratt, V.R. Fast pattern matching in strings // SIAM Journal on Computing. — 1977. — Vol. 6. — № 2. — P. 323–350.
7. Aho, A.V., Corasick, M.J. Efficient string matching: an aid to bibliographic search // Communications of the ACM. — 1975. — Vol. 18. — № 6. — P. 333–340.